

Data Structures and Algorithms – Spring 2023 Final Exam

Allowed Time: 150 minutes

Total Marks: 800

Question 1:

C++ code is REQUIRED for this question

Implement the following recursive method. Do not use any local variables or loops.

```
void pyramidPattern(int n, int x)
```

The output consists of lines of integers. The first line is the number n . The following line is the number $n \ n+1$. The following line is the number $n \ n+1 \ n+2$, and so on, until you reach a line with $n \times x$ numbers. This list of numbers is then repeated backward until you get back to n .

Example output with $n = 1, x = 4$:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
1 2 3 4
1 2 3
1 2
1
```

Question 2:

C++ code is REQUIRED for this question

Using `unordered_map`, find and display the frequency of each distinct number in an input text file "numbers.txt". Each number in the file is separated by a space, tab, or a newline.

Question 3:

C++ code is REQUIRED for this question

Suppose we have a doubly circular linked list having a tail pointer only with integer data. Write a new method for this class to meet the following specification.

```
void decrementEveryNode(Node* tail)
```

```
// Every linked list node has its value decreased by one.
```

Question 4:

C++ code is REQUIRED for this question

Implement the Heap Sort algorithm to sort the array in ascending order.

Question 5:

C++ code is REQUIRED for this question

Implement the Binary Search algorithm to find a target value in the sorted array. The function should return the index of the target if it exists in the array or -1 if the target is not found.

Question 6:

C++ code is REQUIRED for this question

Write a recursive function to compute the height of a Binary Search Tree. The height of a binary tree is the number of nodes on the longest path in the tree.

Question 7:

C++ code is REQUIRED for this question

Write a function to find the k^{th} smallest element in a Binary Search Tree (BST).

Question 8:

Code is NOT REQUIRED for this question

Consider the values given below:

40	20	25	30	35	100	170	140	190	90	80
----	----	----	----	----	-----	-----	-----	-----	----	----

Using these values, create (draw on paper) a self-balancing tree that you have studied in your DSA course. All values must be inserted in the given order.

- Add 40, then 20, 25, 30, 35, 100, 170, 140, 190, 90, and 80.
- All intermediate steps of insertion should be shown.